

GAME PROGRAMMING COURSE — UNITY 2D

カメラ演出を作ろう

シングルトンで作る CameraManager クラス

対象：高等部2年（全2コマ）

CAME
RA
WORK

LESSON 1

シングルトンと追従カメラ

目標：どこからでも呼べるカメラの土台を作り、プレイヤーを追従させる

完成イメージ：1行で呼べるカメラ

C# どこかのスクリプト

```
1 // 追従する相手を決める
2 CameraManager.Instance.SetTarget(player);
3
4 // ボスに2秒注目して、プレイヤーに戻る
5 CameraManager.Instance.Focus(boss, 2.0f);
6
7 // 攻撃が当たったとき
8 CameraManager.Instance.Shake(0.3f, 0.2f);
9 CameraManager.Instance.PunchZoom(0.85f, 0.15f);
```

SetTarget()

追従する相手を設定

SetBounds()

映す範囲を制限

Shake()

画面を揺らす

PunchZoom()

一瞬だけ寄る

Focus()

別の対象に注目

みんながカメラを直接動かす

C# あちこちのスクリプト

```
1 // PlayerMove.cs ... 毎フレーム自分を映す
2 Camera.main.transform.position = new Vector3(pos.x, pos.y, -10f);
3
4 // Enemy.cs ... 攻撃が当たったら揺らす
5 Camera.main.transform.position += (Vector3)Random.insideUnitCircle * 0.3f;
6
7 // Boss.cs ... 登場したら自分を映す
8 Camera.main.transform.position = new Vector3(bossPos.x, bossPos.y, -10f);
```

POINT

それぞれのスクリプトが **Camera.main** の位置を直接書き換えている。
一見動きそうだが、組み合わせると問題が出る。

何が困るのか

① 取り合いになる

Enemy が揺らしても、次のフレームで PlayerMove が上書きする。揺れもボス演出も消えてしまう。

② 同じ処理がバラバラ

$z = -10$ や揺れ方の計算が、いろいろなスクリプトにコピーされている。

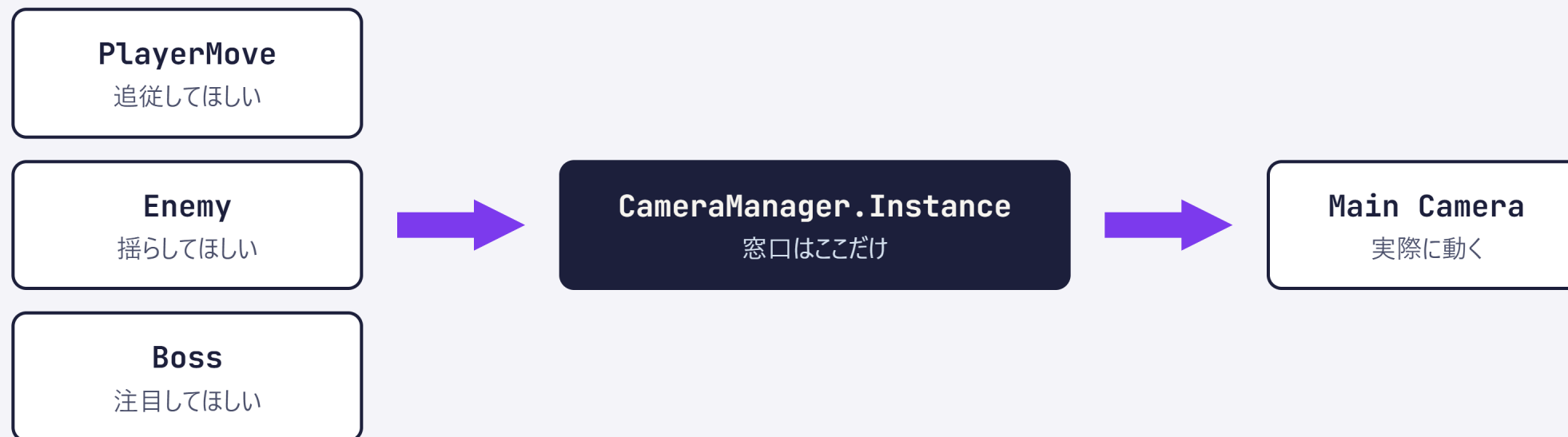
③ 直す場所が多い

「カメラを少しゆっくり追従させたい」だけで、全スクリプトを探して直すことになる。

 カメラを動かすのは **CameraManager** の1か所だけにする

シングルトンとは

そのクラスのインスタンスが**1つ**だけであることを保証し、
どこからでもそのインスタンスにアクセスできるようにする仕組み。



シングルトンの注意点

便利なところ

- どこからでも1行で呼べる
- インспекタで参照をつなぐ手間がない
- カメラのように「1つしかないもの」と相性がよい

危ないところ

- どこからでも変更できるので、バグの犯人を探しにくい
- 便利なので何でも入れたくなる → 何でも抱えた「神クラス」になる
- シーンに2つあると壊れる → 2つ目は消す

ルール：CameraManager に入れるのは「カメラのこと」だけ

シングルトンの基底クラス

シングルトンの仕組みは毎回書かずに、用意してある基底クラスを継承して使う。

SceneSingleton<T>

- シーンを切り替えと一緒に破棄される
- 見つからなくても新しく作らない (null)
- 向いているもの：カメラ、ステージの UI など

PersistentSingleton<T>

- シーンを切り替えても残る (DontDestroyOnLoad)
- 見つからなければ自動で作る
- 向いているもの：サウンド、セーブデータなど

POINT

Main Camera はシーンごとに置かれるので、CameraManager は **SceneSingleton** を継承する。

SceneSingleton.cs ① Instance の取り出し

```
C# SceneSingleton.cs
1  using UnityEngine;
2
3  // シーンを切り替えると一緒に破棄されるシングルトン
4  public class SceneSingleton<T> : MonoBehaviour where T : Component
5  {
6      private static T instance;
7
8      public static T Instance
9      {
10         get
11         {
12             // 見つからなくても新しくは作らない (null が返る)
13             if (instance == null)
14             {
15                 instance = FindAnyObjectByType<T>();
16             }
17             return instance;
18         }
19     }
```

POINT

T には継承したクラス (CameraManager) が入る。最初に使ったとき、シーンから探して覚える。

SceneSingleton.cs ② 登録と後片付け

C# SceneSingleton.cs (続き)

```
20     protected virtual void Awake()
21     {
22         if (instance == null) { instance = this as T; }
23         else if (instance != this) { Destroy(gameObject); } // 2つ目は消す
24     }
25
26     protected virtual void OnDestroy()
27     {
28         // シーン破棄で自分が消えたら、登録を外す
29         if (instance == this) { instance = null; }
30     }
31 }
```

where T : Component

T の条件

T はコンポーネント (MonoBehaviour など) に限る、という約束。

this as T

自分を T として登録

基底クラスから見た自分を、CameraManager 型に変換して覚える。

protected virtual

子クラスで上書きできる

継承したクラスから override できる。外からは呼べない。

Unity の準備

手順	作業	メモ
1	2D のプロジェクトを開く	Unity 6 / 2D テンプレート
2	SceneSingleton.cs が Scripts フォルダにあるか確認	なければ配布したものを入れる
3	Scripts フォルダに CameraManager.cs を作る	クラス名とファイル名を合わせる
4	Main Camera に CameraManager をアタッチ	Camera と同じオブジェクトに付ける
5	Camera の Projection を確認	Orthographic（2D は最初からこれ）
6	動かせる Player を置く	移動スクリプトは既存のものでよい
7	CameraManager の Target に Player をドラッグ	コードを書いた後でOK

コード① クラスの骨組み（シングルトンを継承）

C# CameraManager.cs

```
1 using System.Collections;
2 using UnityEngine;
3
4 public class CameraManager : SceneSingleton<CameraManager>
5 {
6     [Header("追従")]
7     [SerializeField] Transform target;
8     [SerializeField] float smoothTime = 0.2f;
9     [SerializeField] Vector2 offset = new Vector2(0f, 1f);
10
11     [Header("移動範囲")]
12     [SerializeField] bool useBounds = false;
13     [SerializeField] Vector2 minPosition = new Vector2(-10f, -5f);
14     [SerializeField] Vector2 maxPosition = new Vector2(10f, 5f);
```

POINT

: SceneSingleton<CameraManager> と書くだけで CameraManager.Instance が使える。
[SerializeField] は private のままインスペクタで調整できるようにする。

コード② Awake を override する

C# CameraManager.cs (続き)

```
1  Camera cam;
2  Vector3 basePosition;    // 追従で決まる位置
3  Vector3 velocity;        // SmoothDamp用
4
5  protected override void Awake()
6  {
7      base.Awake();          // シングルトンの登録（基底クラス）
8      if (Instance != this) { return; } // 2つ目なら何もしない
9
10     cam = GetComponent<Camera>();
11     basePosition = transform.position;
12 }
```

POINT

基底クラスの Awake は **protected virtual**。override して、最初に必ず **base.Awake()** を呼ぶ。

継承して使うときの3つのルール

`base.Awake();`

① 最初に必ず呼ぶ

呼び忘れると Instance に登録されず、2つ目の削除も行われない。

`if (Instance  $\neq$  this) return;`

② 消える側は準備しない

2つ目の CameraManager は基底クラスが Destroy する。その後の初期化は不要。

`OnDestroy` は書かない

③ 後片付けは基底クラス

登録の解除は SceneSingleton がやる。自分で書くなら override して `base.OnDestroy()` を呼ぶ。

POINT

基底クラスにシングルトンの仕組みをまとめておけば、CameraManager には**カメラの処理**だけを書けばよい。

ポイント

なぜ LateUpdate で動かすのか

1フレームの中の順番

Update

プレイヤーや敵が動く



LateUpdate

カメラが追いかける



描画

画面に表示される

NG : Update で追従

プレイヤーより先にカメラが動くと、1フレーム古い位置を映す。実行順しだいで画面がガタガタする。

OK : LateUpdate で追従

全員の Update が終わった「後」に呼ばれる。必ず最新の位置を映せる。

コード④ SmoothDamp で追従

C# CameraManager.cs (続き)

```
1  void LateUpdate()
2  {
3      if (target != null)
4      {
5          Vector3 goal = target.position + (Vector3)offset;
6          goal.z = basePosition.z;
7          basePosition = Vector3.SmoothDamp(basePosition, goal, ref velocity, smoothTime);
8      }
9
10     transform.position = basePosition;
11 }
12
13 public void SetTarget(Transform newTarget)
14 {
15     target = newTarget;
16 }
```

POINT

ここで一度再生してみよう。Player を動かすと、カメラが少し遅れてついてくる。

追従コードの3つのポイント

① `goal.z = basePosition.z`

カメラは $z = -10$ 、プレイヤーは $z = 0$ 。 z までそのまま追うと、カメラがキャラと同じ面に来て何も映らない。

② `SmoothDamp(今, 目標, ref 速度, 時間)`

「今の速度」は関数の中で書き換わるので `ref` で渡す。だから `velocity` を変数で持っておく。

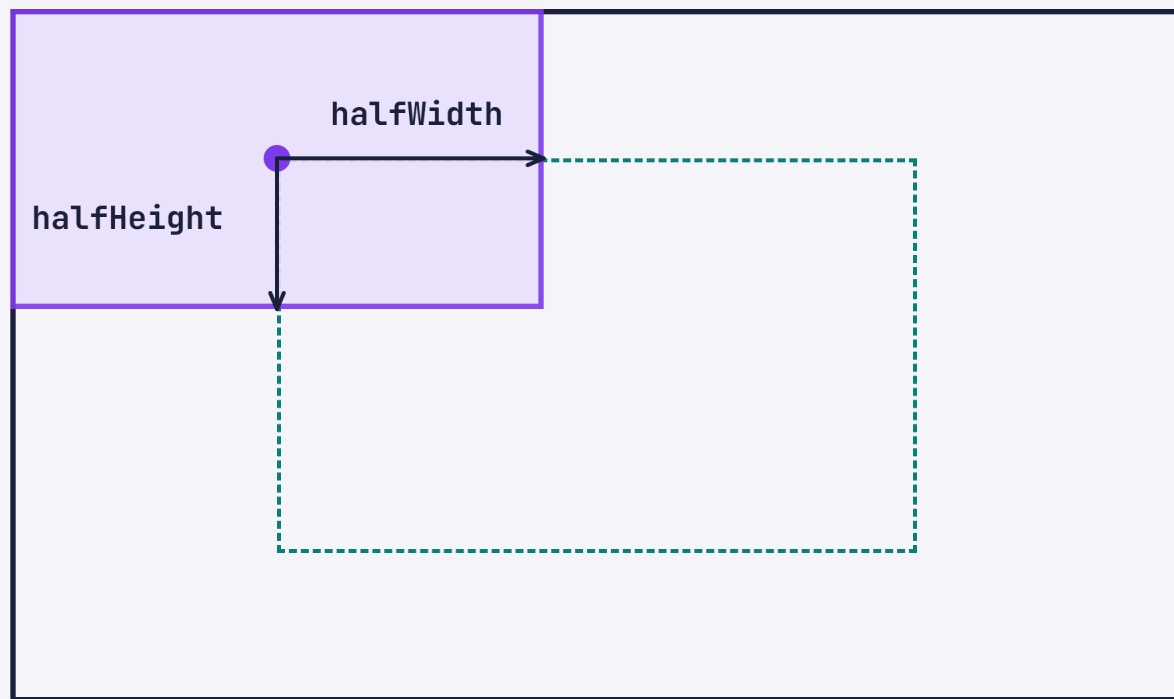
③ `basePosition` を別に持つ

`transform.position` とは別に「本来の位置」を覚えておく。LESSON 2 でシェイクを足すための準備。

smoothTime の違い

値	動き
0	ぴったり張り付く
0.1	キビキビ
0.2	ほどよい（初期値）
0.5	ふんわり
1.0	遅すぎて置いていかれる

移動範囲の制限



- 黒：ステージ
- 紫：画面に映る範囲
- 緑の点線：カメラの中心が動ける範囲

カメラの**中心**が動ける範囲は、
ステージから**画面の半分だけ**
内側になる。

```
halfHeight = orthographicSize  
halfWidth = halfHeight × aspect
```

orthographicSize は「画面の縦の半分」。
aspect は画面の横÷縦（16:9 なら約1.78）。

コード⑤ 移動範囲の制限

C# CameraManager.cs (続き)

```
1 public void SetBounds(Vector2 min, Vector2 max)
2 {
3     minPosition = min;
4     maxPosition = max;
5     useBounds = true;
6 }
7
8 Vector3 ClampToBounds(Vector3 pos)
9 {
10     float halfHeight = cam.orthographicSize;
11     float halfWidth = halfHeight * cam.aspect;
12
13     pos.x = Mathf.Clamp(pos.x, minPosition.x + halfWidth, maxPosition.x - halfWidth);
14     pos.y = Mathf.Clamp(pos.y, minPosition.y + halfHeight, maxPosition.y - halfHeight);
15     return pos;
16 }
```

POINT

Mathf.Clamp(値, 最小, 最大) : 値を最小～最大の間に収める。はみ出したら端の値になる。

コード⑥ LateUpdate に制限を追加

C# CameraManager.cs (LateUpdate を変更)

```
1  void LateUpdate()
2  {
3      if (target != null)
4      {
5          Vector3 goal = target.position + (Vector3)offset;
6          goal.z = basePosition.z;
7          basePosition = Vector3.SmoothDamp(basePosition, goal, ref velocity, smoothTime);
8      }
9
10     if (useBounds) // ← 追加
11     {
12         basePosition = ClampToBounds(basePosition);
13     }
14
15     transform.position = basePosition;
16 }
```

POINT

位置を計算した「後」、transform に入れる「前」に制限をかける。

追従カメラを調整しよう

1

smoothTime を変えて比べる

0 / 0.1 / 0.5 に変えて再生し、動きの違いを確かめよう。

2

ステージの端を設定する

ステージの左下・右上に合わせて Min Position / Max Position を入力し、Use Bounds をオンにしよう。

3

追従対象を切り替える

Cキーを押すと、追従する相手が別のオブジェクトに切り替わるようにしよう。

ヒント : `CameraManager.Instance.SetTarget(other);`

LESSON 2

演出を足す

目標：シェイク・瞬間ズーム・注目カメラを CameraManager に追加する

位置は「足し算」で決める

basePosition

追従で決まる位置

+

shakeOffset

シェイクのずれ

=

transform.position

実際のカメラの位置

NG：シェイクが position を直接書き換える

揺らしても、次のフレームで追従が上書きしてしまう。揺れが終わった後、どこに戻ればいいかもわからない。

OK：別々に計算して最後に足す

追従とシェイクがおたがいを邪魔しない。シェイクが終わったら shakeOffset を 0 に戻すだけ。

transform.position に代入するのは LateUpdate の最後の1回だけ、がルール。

コルーチン

書き方	意味
StartCoroutine(Method())	開始する。戻り値の Coroutine を変数に保存できる
yield return null;	1フレーム待つ
yield return new WaitForSeconds(2f);	2秒待つ
StopCoroutine(保存した変数)	途中で止める

「○秒かけて変化させる」基本の形

C# 基本パターン

```
1 float time = 0f;
2 while (time < duration)
3 {
4     // ここで少しずつ変化させる
5     time += Time.deltaTime;
6     yield return null;
7 }
```

コード⑦ LESSON 2 で使う変数を追加

C# CameraManager.cs (変数に追加)

```
1      enum CameraMode { Follow, Focus }
2
3      CameraMode mode = CameraMode.Follow;
4      Transform focusTarget;
5
6      Vector3 shakeOffset;      // シェイクのずれ
7      float defaultSize;      // 元のカメラサイズ
8
9      Coroutine shakeRoutine;
10     Coroutine zoomRoutine;
11     Coroutine focusRoutine;
```

C# CameraManager.cs (Awake に追加)

```
1      defaultSize = cam.orthographicSize;    // ← Awake の最後に追加
```

POINT

Coroutine を変数に入れておくのは、前の演出を止めてから新しい演出を始めるため。

コード⑧ カメラシェイク

C# CameraManager.cs (続き)

```
1 public void Shake(float power, float duration)
2 {
3     if (shakeRoutine != null)
4     {
5         StopCoroutine(shakeRoutine);
6     }
7     shakeRoutine = StartCoroutine(ShakeRoutine(power, duration));
8 }
9
10 IEnumerator ShakeRoutine(float power, float duration)
11 {
12     float time = 0f;
13     while (time < duration)
14     {
15         float rate = 1f - time / duration; // だんだん弱くする
16         shakeOffset = Random.insideUnitCircle * power * rate;
17         time += Time.deltaTime;
18         yield return null;
19     }
20     shakeOffset = Vector3.zero;
21 }
```

シェイクのしくみ

Random.insideUnitCircle

半径1の円の中のランダムな点（Vector2）。
power をかけると揺れの大きさになる。

rate = 1 - time / duration

最初は 1、終わりに近づくほど 0 に近づく。だんだん弱く揺れる。

StopCoroutine してから開始

連打しても揺れが重ならない。いつも最新のシェイクが1つだけ動く。

最後に、LateUpdate でずれを足す

C# CameraManager.cs (LateUpdate を変更)

```
1 transform.position = basePosition + shakeOffset; // ← LateUpdate の最後を変更
```

POINT

ここで Shake() を呼んでみよう。プレイヤーを追いかけて揺れれば成功。

コード⑨ 瞬間ズーム

C# CameraManager.cs (続き)

```
1 public void PunchZoom(float rate, float duration)
2 {
3     if (zoomRoutine != null)
4     {
5         StopCoroutine(zoomRoutine);
6     }
7     zoomRoutine = StartCoroutine(PunchZoomRoutine(rate, duration));
8 }
9
10 IEnumerator PunchZoomRoutine(float rate, float duration)
11 {
12     float time = 0f;
13     while (time < duration)
14     {
15         float t = time / duration;
16         cam.orthographicSize = Mathf.Lerp(defaultSize * rate, defaultSize, t);
17         time += Time.deltaTime;
18         yield return null;
19     }
20     cam.orthographicSize = defaultSize;
21 }
```

瞬間ズームのしくみ

2D のズーム = orthographicSize

画面の縦の半分の大きさ。小さくすると寄る、大きくすると引く。

Mathf.Lerp(a, b, t)

t が 0 のとき a、1 のとき b。間になめらかにつながる。

一瞬で寄って、ゆっくり戻る

最初のフレームで defaultSize × rate まで寄り、duration 秒かけて元のサイズに戻る。

例：defaultSize = 5、rate = 0.85

t	サイズ
0（開始）	4.25（寄る）
0.5	4.625
1（終了）	5（元どおり）

rate を 1 より大きくすると、一瞬「引く」演出になる。

コード⑩ 注目カメラ

C# CameraManager.cs (続き)

```
1 public void Focus(Transform newTarget, float duration)
2 {
3     if (focusRoutine != null)
4     {
5         StopCoroutine(focusRoutine);
6     }
7     focusRoutine = StartCoroutine(FocusRoutine(newTarget, duration));
8 }
9
10 IEnumerator FocusRoutine(Transform newTarget, float duration)
11 {
12     focusTarget = newTarget;
13     mode = CameraMode.Focus;
14
15     yield return new WaitForSeconds(duration);
16
17     mode = CameraMode.Follow;
18     focusTarget = null;
19 }
```

POINT

モードを Focus にして、決めた秒数待ってから Follow に戻すだけ。動かすのは LateUpdate の仕事。

コード⑪ LateUpdate を注目に対応させる

C# CameraManager.cs (LateUpdate を変更)

```
1 void LateUpdate()
2 {
3     // 今追いかける相手を決める
4     Transform current = target;
5     if (mode == CameraMode.Focus)
6     {
7         current = focusTarget;
8     }
9
10    if (current != null) // target → current
11    {
12        Vector3 goal = current.position + (Vector3)offset; // target → current
13        goal.z = basePosition.z;
14        basePosition = Vector3.SmoothDamp(basePosition, goal, ref velocity, smoothTime);
15    }
16    // (ここから下は今までと同じ)
```

POINT

SmoothDamp のままなので、注目先へもスーッと移動し、戻るときもなめらかに戻る。

コード⑫ 動作確認用スクリプト

C# CameraTest.cs

```
1  using UnityEngine;
2  using UnityEngine.InputSystem;
3
4  public class CameraTest : MonoBehaviour
5  {
6      [SerializeField] Transform boss;
7
8      void Update()
9      {
10         // Zキー：攻撃が当たった演出
11         if (Keyboard.current.zKey.wasPressedThisFrame)
12         {
13             CameraManager.Instance.Shake(0.3f, 0.2f);
14             CameraManager.Instance.PunchZoom(0.85f, 0.15f);
15         }
16         // Xキー：ボスに2秒注目
17         if (Keyboard.current.xKey.wasPressedThisFrame)
18         {
19             CameraManager.Instance.Focus(boss, 2.0f);
20         }
21     }
22 }
```

Unity で試す

手順	作業	確認すること
1	空の GameObject「CameraTest」を作り、CameraTest.cs をアタッチ	エラーが出ていない
2	ボス役のスプライトを置き、Boss 欄にドラッグ	Player から離して置く
3	再生して Z キー	揺れながら一瞬寄る
4	再生して X キー	ボスへ移動し、2秒後に戻る
5	Z を連打する	揺れが重ならない



Input System を使っていないプロジェクトでは、**Input.GetKeyDown(KeyCode.Z)** に置き換える。

演出を作り込もう

1

場面ごとに揺れを変える

ダメージを受けたら強く長く、着地したら弱く短く揺れるように、数値を調整しよう。

2

注目中はズームアウトする

ZoomTo(サイズ, 秒) メソッドを自作し、Focus 中は少し引いた画面にしよう。

3

ポーズ中でも揺れるようにする

Time.timeScale = 0 にしてもシェイクが動くようにしよう。

ヒント : `Time.unscaledDeltaTime`

4

発展：先読みカメラ

プレイヤーの進行方向の少し先を映すカメラにしよう。